Applications, Advantages and Disadvantages of Hash Data Structure

Last Updated : 28 Mar, 2023

Introduction :

Imagine a giant library where every book is stored in a specific shelf, but instead of searching through endless rows of shelves, you have a magical map that tells you exactly which shelf your book is on. That's exactly what a Hash data structure does for your data!

Hash data structures are a fundamental building block of computer science and are used in a wide range of applications such as databases, caches, and programming languages. They are a way to map data of any type, called keys, to a specific location in memory called a bucket. These data structures are incredibly fast and efficient, making them a great choice for large and complex data sets.

Whether you're building a database, a cache, or a programming language, Hash data structures are like a superpower for your data. They allow you to perform basic operations like insertions, deletions, and lookups in the blink of an eye, and they're the reason why your favorite apps and websites run so smoothly.

A hash data structure is a type of data structure that allows for efficient insertion, deletion, and retrieval of elements. It is often used to implement associative arrays or mappings, which are data structures that allow you to store a collection of key-value pairs.

In a hash data structure, elements are stored in an array, and each element is associated with a unique key. To store an element in a hash, a hash function is applied to the key to generate an index into the array where the element will be stored. The hash function should be designed such that it distributes the elements evenly across the array, minimizing collisions where multiple elements are assigned to the same index.

When retrieving an element from a hash, the hash function is again applied to the key to find the index where the element is stored. If there are no collisions, the

element can be retrieved in constant time, O(1). However, if there are collisions, multiple elements may be assigned to the same index, and a search must be performed to find the correct element.

To handle collisions, there are several strategies that can be used, such as chaining, where each index in the array stores a linked list of elements that have collided, or open addressing, where collisions are resolved by searching for the next available index in the array.

Hash data structures have many applications in computer science, including implementing symbol tables, caches, and databases. They are especially useful in situations where fast retrieval of elements is important, and where the number of elements to be stored may be large.

Collision Resolution: Collision resolution in hash can be done by two methods:

- Open addressing and
- Closed addressing.

Open Addressing: Open addressing collision resolution technique involves generating a location for storing or searching the data called **probe**. It can be done in the following ways:

- **Linear Probing:** If there is a collision at **i** then we use the hash function - **H(k, i) = [H'(k) + i] % m**
  where, **i** is the index, **m** is the size of hash table **H(k, i)** and **H'(k)** are hash functions.
- **Quadratic Probing:** If there is a collision at **i** then we use the hash function - **$H(k, i) = [H'(k) + c_1 * i + c_2 * i^2] \% m$**
  where, **i** is the index, **m** is the size of hash table **H(k, i)** and **H'(k)** are hash functions, **c_1** and **c_2** are constants.
- **Double Hashing:** If there is a collision at **i** then we use the hash function - **$H(k, i) = [H_1(k, i) + i * H_2(k)] \% m$**
  where, **i** is the index, **m** is the size of hash table **H(k, i)**, **$H_1(k) = k \% m$** and **$H_2(k) = k \% m'$** are hash functions.

Closed Addressing:

Closed addressing collision resolution technique involves chaining. Chaining in the hashing involves both array and linked list. In this method, we generate a probe with the help of the hash function and link the keys to the respective index one after the other in the same index. Hence, resolving the collision.

Applications of Hash:

- Hash is used in databases for indexing.
- Hash is used in disk based data structures.
- In some programming languages like Python, JavaScript hash is used to implement objects.
- Hash tables are commonly used to implement caching systems
- Used in various cryptographic algorithms.
- Hash tables are used to implement various data structures.
- Hash tables are used in load balancing algorithms
- **Databases:** Hashes are commonly used in databases to store and retrieve records quickly. For example, a database might use a hash to index records by a unique identifier such as a social security number or customer ID.
- **Caches:** Hashes are used in caches to quickly look up frequently accessed data. A cache might use a hash to store recently accessed data, with the keys being the data itself and the values being the time it was accessed or other metadata.
- **Symbol tables:** Hashes are used in symbol tables to store key-value pairs representing identifiers and their corresponding attributes. For example, a compiler might use a hash to store the names of variables and their types.
- **Cryptography:** Hashes are used in cryptography to create digital signatures, verify the integrity of data, and store passwords securely. Hash functions are designed such that it is difficult to reconstruct the original data from the hash, making them useful for verifying the authenticity of data.
- **Distributed systems:** Hashes are used in distributed systems to assign work to different nodes or servers. For example, a load balancer might use a hash to distribute incoming requests to different servers based on the request URL or other criteria.
- **File systems:** Hashes are used in file systems to quickly locate files or data blocks. For example, a file system might use a hash to store the locations of files

on a disk, with the keys being the file names and the values being the disk locations.

Real-Time Applications of Hash:

- Hash is used for cache mapping for fast access of the data.
- Hash can be used for password verification.
- Hash is used in cryptography as a message digest.

Applications of Hash::

- Hash provides better synchronization than other data structures.
- Hash tables are more efficient than search trees or other data structures.
- Hash provides constant time for searching, insertion and deletion operations on average.
- Hash tables are space-efficient.
- Most Hash table implementation can automatically resize itself.
- Hash tables are easy to use.
- Hash tables offer a high-speed data retrieval and manipulation.
- **Fast lookup:** Hashes provide fast lookup times for elements, often in constant time O(1), because they use a hash function to map keys to array indices. This makes them ideal for applications that require quick access to data.
- **Efficient insertion and deletion:** Hashes are efficient at inserting and deleting elements because they only need to update one array index for each operation. In contrast, linked lists or arrays require shifting elements around when inserting or deleting elements.
- **Space efficiency:** Hashes use space efficiently because they only store the key-value pairs and the array to hold them. This can be more efficient than other data structures such as trees, which require additional memory to store pointers.
- **Flexibility:** Hashes can be used to store any type of data, including strings, numbers, and objects. They can also be used for a wide variety of applications, from simple lookups to complex data structures such as databases and caches.
- **Collision resolution:** Hashes have built-in collision resolution mechanisms to handle cases where two or more keys map to the same array index. This

ensures that all elements are stored and retrieved correctly.

Disadvantages of Hash:

- Hash is inefficient when there are many collisions.
- Hash collisions are practically not be avoided for large set of possible keys.
- Hash does not allow null values.
- Hash tables have a limited capacity and will eventually fill up.
- Hash tables can be complex to implement.
- Hash tables do not maintain the order of elements, which makes it difficult to retrieve elements in a specific order.

Next Article
Applications, Advantages and Disadvantages of Hash Data Structure

Similar Reads

Applications, Advantages and Disadvantages of Trie

Trie(pronounced as "try"): Trie(also known as the digital tree or prefix tree) is a sorted and efficient tree-based special data structure that is used to store and retrieve keys in a dataset of strings. The basic...

4 min read

Applications, Advantages and Disadvantages of Set

In this article, we will unlock the potential of your data with the elegance and efficiency of Set Data Structures. A set is a collection of unique elements, it's a mathematical concept that has been...

3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

2 min read

Applications, Advantages and Disadvantages of String

The String data structure is the backbone of programming languages and the building blocks of communication. String data structures are one of the most fundamental and widely used tools in...

6 min read

Applications, Advantages and Disadvantages of Binary Tree

A binary tree is a tree that has at most two children for any of its nodes. There are several types of binary trees. To learn more about them please refer to the article on "Types of binary tree" Applications:Genera...

2 min read

Applications, Advantages and Disadvantages of Array

Array is a linear data structure that is a collection of data elements of same types. Arrays are stored in contiguous memory locations. It is a static data structure with a fixed size.Table of ContentApplications ...

2 min read

Applications, Advantages and Disadvantages of Linked List

A Linked List is a linear data structure that is used to store a collection of data with the help of nodes. Please remember the following points before moving forward.The consecutive elements are connected ...

4 min read

Applications, Advantages and Disadvantages of Circular Linked List

A Linked List is a popular data structure which is used to store elements. It is a linear data structure. It contains nodes that have a pointer to store the address of the next node and data which is the value of...

6 min read

Advantages and Disadvantages of Tree

Tree is a non-linear data structure. It consists of nodes and edges. A tree represents data in a hierarchical organization. It is a special type of connected graph without any cycle or circuit.Advantages of...

2 min read

Real-life Applications of Data Structures and Algorithms (DSA)

You may have heard that DSA is primarily used in the field of computer science. Although DSA is most commonly used in the computing field, its application is not restricted to it. The concept of DSA can also...

10 min read